

App Dev Project Report (MAD-1)

Student Details

Name: Ritik Roshan

Roll Number: 24f1000379

Email: 24f1000379@ds.study.iitm.ac.in

About Me:

I'm currently enrolled in the IIT Madras BS Degree Program, and I have a strong interest in software development, databases, and full-stack engineering. I enjoy creating structured, practical systems that solve real-world problems and make processes more efficient for users. This project gave me hands-on experience with Flask, routing, authentication, and database modeling, and helped solidify my understanding of how complete web applications are built.

Project Details

Project Title:

Hospital Management System (HMS)

Problem Statement:

Many hospitals still rely on manual logs or outdated tools to manage doctors, patients, appointments, and treatment history. This often leads to scheduling errors, difficulty accessing records, and an overall inefficient workflow.

The goal of this project was to build a complete web-based Hospital Management System using Flask, Jinja2, and SQLite. The system supports:

- Admin functionalities
- Doctor functionalities
- Patient functionalities

Each user type has a dedicated dashboard and access controls enforced through role-based authentication.

Approach:

The project is built using Flask with Blueprints to organize routes for admin, doctor, patient, and authentication modules. The database is handled through SQLAlchemy ORM and is generated programmatically.

User interfaces are designed with Bootstrap 5 for a clean, responsive layout.

System flow:

1. Admin (pre-created) logs in → manages doctors, patients, and appointments
2. Doctor logs in → views appointments → updates treatment history
3. Patient registers → searches doctors → books appointments → views their history

Role-based access control ensures users only see features relevant to them.

AI / LLM Declaration

AI tools (ChatGPT – GPT-5 family) were used **only for assistance**, such as:

- *Suggesting folder structures*
- *Improving UI/UX for Bootstrap templates*
- *Debugging Flask and SQLAlchemy issues*
- *Writing sample YAML API documentation*
- *Improving naming consistency*

AI Usage Extent: ~20–25%

Only partial guidance and suggestions were taken—no complete modules were generated. All logic, implementation, debugging, and integration were done manually.

I can independently explain:

- Blueprints and routing
 - Database design and relationships
 - Authentication using Flask-Login
 - Jinja2 rendering
 - CRUD operations
 - Appointment conflict handling
-

Technologies & Frameworks Used

Technology	Purpose
Flask	Backend framework, routing, controllers
Flask-Login	Authentication, session & role management
SQLAlchemy	ORM, schema, database operations
SQLite	Lightweight database
Jinja2	Template rendering
Bootstrap 5	UI design & responsiveness
WTForms	Form validation
JavaScript	Interactivity

Technology Purpose

Chart.js (opt.) Appointment visualization

Database Schema / ER Diagram Description

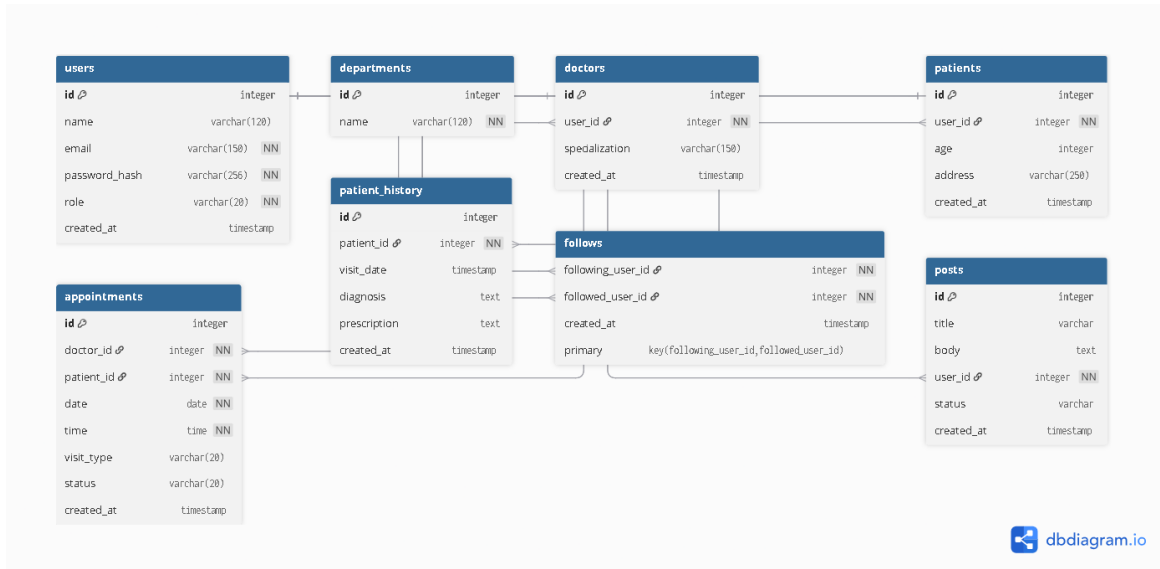
Tables

1. **User**
id, name, email, password, role (admin/doctor/patient)
2. **Doctor**
id, name, specialization, experience, user_id (FK)
3. **Patient**
id, name, age, gender, contact, user_id (FK)
4. **Appointment**
id, patient_id, doctor_id, date, time, status
5. **TreatmentHistory**
id, appointment_id, visit_type, diagnosis, prescription, medicines
6. **DoctorAvailability**
id, doctor_id, date, slot1–slot4 (prevents double booking)
7. **Department**
id, name, description

Relationships

- User ↔ Doctor : One-to-One
- User ↔ Patient : One-to-One
- Doctor ↔ Appointment : One-to-Many
- Patient ↔ Appointment : One-to-Many
- Appointment ↔ TreatmentHistory : One-to-One

- Doctor ↔ DoctorAvailability : One-to-Many



API Resource Endpoints (Optional)

1. Common / Authentication API Routes

These routes handle login, logout, and user verification.

Endpoint	Method	Description
/api/login	POST	Authenticate user (admin/doctor/patient)
/api/logout	POST	Logout current user
/api/register	POST	Patient registration only (doctors & admin can't self-register)
/api/user	GET	Returns current logged-in user details
/api/change-password	PUT	Change password for any user

2. Admin Module API Endpoints

Admin manages doctors, patients, and appointments.

Doctors Management

Endpoint	Method	Description
/api/admin/doctors	GET	Get all doctors
/api/admin/doctor/add	POST	Add new doctor
/api/admin/doctor/<id>	GET	Get one doctor details

Endpoint	Method	Description
/api/admin/doctor/<id>/update	PUT	Update doctor profile
/api/admin/doctor/<id>/delete	DELETE	Remove/blacklist doctor

3. Doctor Module API Endpoints

Doctor Appointment Management

Endpoint	Method	Description
/api/doctor/<id>/appointments	GET	Get doctor's upcoming appointments
/api/doctor/appointment/<id>/complete	PUT	Mark appointment completed
/api/doctor/appointment/<id>/cancel	PUT	Cancel appointment
/api/doctor/appointment/<id>/treatment	POST	Add treatment details
/api/doctor/patient/<id>/history	GET	Get full medical history of patient
/api/doctor/<id>/availability	GET	Get 7-day availability
/api/doctor/<id>/availability/update	POST	Update availability slots

4. Patient Module API Endpoints

Profile Management

Endpoint	Method	Description
/api/patient/<id>	GET	Get patient profile
/api/patient/<id>/update	PUT	Update profile
/api/patient/doctors	GET	List all doctors
/api/patient/doctors/search	GET	Search by specialization/name
/api/patient/doctor/<id>/availability	GET	View doctor availability

YAML file: Provided separately as api.yaml.

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy
- **/routes** – Flask Blueprints for user and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS, JS, and chart visualization files

Implemented Features:

User registration and login
Daily activity tracking and habit logging
Visualization dashboard using Chart.js
Summary analytics with date filtering
Responsive UI built with Bootstrap
Secure password hashing and user session handling

Video Presentation

Drive Link:

(<https://drive.google.com/file/d/1ARQRRgRGVYi2KTBCexfVmgehJN26irHt/view?usp=sharing>)